

CSE 312

İşletim Sistemleri

Dosya Sistemleri

File Systems

Kapsamlı Konu Anlatımı & Soru Çözümleri

Spring 2026

- Dosya Sistemi Tasarım Kararları

- Dosya Yapıları ve Metadata

- Disk Alan Tahsisi Yöntemleri (Contiguous / Linked / Indexed)

- I-node Yapısı ve Unix Dosya Sistemi

- Dizinler ve Paylaşılan Dosyalar

- Journaling ve Çökme Kurtarma

- Sanal Dosya Sistemi (VFS)

- Disk Alan Yönetimi ve Performans

- Yedekleme Stratejileri

- Quiz Soruları ve Çözümleri

İçindekiler

1	Dosya Sistemlerine Giriş	3
2	Tasarım Kararları (Design Decisions)	4
3	Dosya Yapıları ve Türleri	6
4	Metadata ve Depolama	8
5	Disk Alan Tahsisi Yöntemleri	10
5 . 1	Bitişik Tahsis (Contiguous Allocation)	10
5 . 2	Bağlı Liste Tahsisi (Linked Allocation)	11
5 . 3	FAT (File Allocation Table)	12
5 . 4	İndeksli Tahsis (Indexed Allocation) ve I-node	13
6	Unix V7 ve Modern Unix Dosya Sistemi	15
7	Dizinler (Directories)	19
8	Paylaşılan Dosyalar ve Hard Link	21
9	Journaling ve Çökme Kurtarma	22
10	Sanal Dosya Sistemi (VFS)	23
11	Disk Alan Yönetimi ve Performans	24
12	Yedekleme (Backup) Stratejileri	26
13	Dosya Sistemi Tutarlılığı (fsck)	28
14	Özel Dosya Sistemleri: ISO 9660, FAT, NTFS	29
15	Quiz Soruları ve Cevapları	31
16	Kitap Bölüm Sonu Soruları ve Çözümleri	35

1. Dosya Sistemlerine Giriş

Dosya sistemi (file system), işletim sisteminin depolama kaynaklarını soyutlayan en görünür bileşenlerinden biridir. Kullanıcılar her gün dosyalarla etkileşime girer; bu etkileşimin altında çok katmanlı bir soyutlama yatmaktadır.

Temel Tanımlar

Dosya (File): İşletim sisteminin depolama kaynakları için sağladığı mantıksal depolama birimidir. Adres uzayının uzantısı (geçici dosyalar) veya kalıcı depolama birimi olabilir.

Dosya Sistemi (File System): Dosya adlarını fiziksel ortamın bir alt kümesine eşleyen bir eşlemedir. İşletim sisteminin depolama kaynakları için soyutlama katmanıdır.

Dizin (Directory): Bir grup dosya için mantıksal bir 'kap' görevi gören yapıdır.

Uzun Vadeli Bilgi Depolamanın Gereksinimleri

Gereksinim	Açıklama
Büyük Kapasite	Çok büyük miktarda bilgi depolanabilmelidir (petabayt düzeyinde).
Kalıcılık (Persistence)	Bilgi, onu kullanan sürecin sonlanmasından sonra da hayatta kalmalıdır.
Eş Zamanlı Erişim	Birden fazla süreç bilgiye aynı anda erişebilmelidir.

Gerçek Dünya Örneği

Dijital bir uzun metrajlı film yaklaşık **2 petabayt** veri üretir (~500K DVD). Her yıl 700 yeni film çekilmektedir. Bir petabaytı 1 Gbit/s Ethernet üzerinden aktarmak **90 gün** sürer. Bu, dosya sistemlerinin neden bu denli kritik tasarım kararları gerektirdiğini açıkça ortaya koymaktadır.

(Kaynak: IEEE Spectrum, Mart 2014)

Disk Soyutlaması

Diski, sabit boyutlu bloklardan oluşan doğrusal bir dizi olarak düşünebiliriz. Temel işlemler blok okuma ve blok yazmadan ibarettir. Bu basit soyutlama üzerinde birçok kritik soru ortaya çıkar:

- Bilgiye nasıl ulaşılır?
- Bir kullanıcının başkasının verisini okuması nasıl engellenir?
- Hangi blokların boş olduğu nasıl bilinir?

2. Tasarım Kararları (Design Decisions)

Bir dosya sistemi tasarlanırken pek çok kritik karar verilmesi gerekir. Bu kararlar birbirini etkileyen karmaşık dengelerden oluşur. Aşağıda temel tasarım boyutları incelenmiştir.

2.1 Dosya İsimlendirme (File Naming)

Dosya isimlendirme, kullanıcı arayüzünün temelidir. Farklı işletim sistemleri farklı kurallar benimser:

İşletim Sistemi	Örnek Dosya Yolu	Özellik
Unix/Linux	/this/is/a/unix.file.name	Büyük/küçük harf duyarlı, kernel extension bilmez
Windows	q:\Windows likes\mOnOcaSE	Büyük/küçük harf duyarsız, 8.3 veya uzun ad
Multics	>Multics>usesup-a-level	Özel path ayırıcı karakterler
VMS	host::device[dir.ory]name.type;1	Sürüm numarası (versiyon) içerir
OS/360	os.360.looks.hierarchical	Görünüşte hiyerarşik ama değil

⚠ Extension (Uzantı) Davranışı

Unix/Multics: Kernel, dosya uzantısından tamamen habersizdir. Uzantı sadece karakterdir. Ancak make, C derleyicisi gibi araçlar uzantıyı yorumlar.

Eski DOS: '8.3' formatı: 8 karakterlik dosya adı + 3 karakterlik uzantı.

OS/360: Uzantıları izin seviyeleriyle karıştırmıştır — tasarım hatası olarak kabul edilir.

2.2 Medya Özellikleri (Media Characteristics)

Medya Türü	Sektör Yapısı	Özellik
Modern HDD	512-byte veya 4096-byte sabit sektörler	Rastgele erişim, blok yazılabilir
Eski mainframe	Değişken boyutlu sektörler	Uygulama seçiyordu
CD/DVD	Sabit blok, yazılabilir versiyonlar	Blok üzerine yazılamaz
Flash (SSD)	Sabit blok	Her blok sınırlı sayıda yazılabilir (wear)
WORM	Sabit blok	Silinebilir ama üzerine yazılamaz

2.3 Versiyonlama (Versioning)

Bazı dosya sistemleri, aynı dosyanın birden fazla sürümünü saklama olanağı sunar:

- **VMS, Tenex, TOPS-20:** Sürüm numarası, dosya adının parçasıdır (örn. file.txt;3).
- **Plan 9:** Herhangi bir geçmiş zaman noktasında dosya sistemi görünümü sunabilir.

2.4 Diğer Kritik Tasarım Soruları

Hiyerarşi	Dosya sistemi hiyerarşik mi yoksa düz mı?
Erişim Kontrolü	Hangi tür erişim denetimi uygulanacak?
Boyut	Dosya sistemi ne kadar büyük/küçük olabilir?
Fault Recovery	Çöküşten sonra tutarlılık nasıl sağlanır?
Yapı	Kayıt-, bayt- veya blok-yapılı mı?
Çoklu İsim	Bir dosyanın birden fazla adı olabilir mi?

3. Dosya Yapıları ve Türleri

Dosyalar farklı yapısal modellerde organize edilebilir. Her model farklı kullanım senaryolarına uygun avantajlar sunar.

3.1 Üç Temel Dosya Yapısı

Yapı Türü	Kullanan Sistemler	Açıklama	Avantaj	Dezavantaj
Bayt Dizisi (Byte Sequence)	Unix/Linux Windows	En esnek yapı. OS, içeriği bilmez; yorumlama uygulamaya bırakılır.	Yüksek esneklik	Yapı yok
Kayıt Dizisi (Record Sequence)	Eski sistemler IBM mainframe	Sabit veya değişken boyutlu kayıtlar. Punch kartı imajlarına dayanır.	Yapılandırılmış	Esneksiz
Ağaç (Tree)	Bazı DB sistemleri	Anahtara göre sıralı erişim sağlar. Veritabanı dosyaları için uygundur.	Hızlı arama	Karmaşık

Unix Felsefesi

Unix'te dosya, rastgele adreslenebilir bir bayt dizisidir. Kernel, dosyanın içeriğinden tamamen habersizdir. Bu tasarım kararı, Unix'in olağanüstü esnekliğinin temel kaynağıdır. 'Her şey bir dosyadır' felsefesi (aygıtlar, dizinler, soketler...) bu soyutlama üzerine inşa edilmiştir.

3.2 Dosya Türleri (File Types)

Unix'te özel dosya türleri bulunur. Bu türler, kernel tarafından farklı biçimde işlenir:

Normal Dosya (Regular File)	Kullanıcı verisi içerir. Metin veya binary olabilir.
Dizin (Directory)	Özel bir dosya türüdür; içinde dosya ve alt dizin listesi bulunur.
Karakter Aygıtı (Character Device)	I/O aygıtlarını modellemek için kullanılır (terminal, fare vb.)
Blok Aygıtı (Block Device)	Diskler gibi blok tabanlı aygıtları modellemek için kullanılır.
Sembolik Link (Symbolic Link)	Başka bir dosyaya/dizine işaret eden yumuşak link.
Pipe / FIFO	Süreçler arası iletişim kanalı.
Socket	Ağ veya yerel IPC için soket dosyası.

3.3 Çalıştırılabilir Dosya (Executable) Yapısı

Bir Unix çalıştırılabilir dosyası (ELF formatı), birkaç temel bölümden oluşur:

[ELF BINARY FORMAT]

Magic number : İşletim sistemi bu dosyanın executable olduğunu anlar
Başlık (Header): Mimari bilgisi, entry point adresi
.text bölümü : Makine kodu (program talimatları)
.data bölümü : Başlatılmış global/statik değişkenler
.bss bölümü : Başlatılmamış global değişkenler
Sembol tablosu: Hata ayıklama için sembol bilgileri

4. Metadata ve Depolama

Metadata (üstveri), dosyanın içeriği dışında dosyayı tanımlayan tüm bilgileri kapsar. Dosya sistemi, her dosya için kapsamlı metadata saklamak zorundadır.

4.1 Metadata İçeriği

Disk Blokları	Dosyanın disk üzerindeki fiziksel konumu
Dosya Boyutu	Bayt cinsinden toplam boyut
Oluşturma Zamanı	Dosyanın ilk oluşturulduğu tarih/saat
Değiştirilme Zamanı (mtime)	İçeriğin son değiştirildiği tarih
Erişim Zamanı (atime)	Son erişim tarih/saati
Inode Değişim Zamanı (ctime)	Metadata'nın son değiştiği tarih
Sahip (Owner)	Dosyayı oluşturan kullanıcının UID'si
Grup (Group)	Dosyanın ait olduğu grup GID'si
İzinler (Permissions)	rw-rw-rw formatında erişim hakları
Hard Link Sayısı	Dosyayı gösteren dizin girdisi sayısı
Dosya Türü	Normal, dizin, sembolik link, aygıt...

4.2 Linux stat Yapısı

Linux'ta her dosyanın metadata'sı **struct stat** ile erişilebilir. Bu yapı, stat() sistem çağrısı ile doldurulur:

```
[ C ]
struct stat {
    dev_t    st_dev;        /* aygıt numarası */
    ino_t     st_ino;       /* inode numarası */
    mode_t    st_mode;      /* dosya türü + izinler */
    nlink_t   st_nlink;     /* hard link sayısı */
    uid_t     st_uid;       /* sahibin kullanıcı ID'si */
    gid_t     st_gid;       /* grup ID'si */
    dev_t     st_rdev;      /* aygıt türü (inode aygıtı ise) */
    off_t     st_size;      /* bayt cinsinden toplam boyut */
    blksize_t st_blksize;   /* dosya sistemi I/O blok boyutu */
    blkcnt_t  st_blocks;    /* tahsis edilen 512-byte blok sayısı */
    time_t    st_atime;     /* son erişim zamanı */
    time_t    st_mtime;     /* son değiştirme zamanı */
    time_t    st_ctime;     /* son durum değişikliği zamanı */
};
```

4.3 Metadata Nerede Saklanır?

Konum	Örnek	Dezavantaj / Avantaj	Kullanım
Dosyanın İçinde	Apple: Resource + Data fork	Taşınabilir değil; kopyalamada kaybolur	Tarihe karışmış
Dizin Girdisinde	Windows FAT	Metadata'ya hızlı erişim; hard link zor	Özel metadata bitleri için
Ayrı Yapıda (inode)	Unix/Linux	Hard link destekler; boyutu denetimli tutulur	Modern tercih
Bölünmüş (Split)	Bazı modern FS	Sık kullanılan metadata hızlı erişimde	Hibrit çözüm

⚠ Dizinde Metadata Tutmanın Avantajları ve Sorunları

Avantaj: Metadata'ya erişmek için ek disk okuması gerekmez — dizin okunduğunda metadata da gelir.

Sorun: Hard link (çoklu isim) desteği güçleşir: Metadata'nın tüm kopyaları senkronize tutulmalıdır.

Modern çözüm: Birçok yeni sistem, ağaç gezintisini hızlandırmak için dizinde yalnızca 'dosya türü' gibi birkaç bit metadata saklar.

5. Disk Alan Tahsisi Yöntemleri

Dosya sistemi, dosyalar için disk bloklarını tahsis etmek ve tahsis edilmiş alanları takip etmek zorundadır. Bu, RAM tahsisine benzer ama disk'e özgü kısıtlamalar içerir. Üç temel tahsis yöntemi vardır.

5.1 Bitişik Tahsis (Contiguous Allocation)

Her dosya, disk üzerinde bitişik (ardışık) blok kümesi kaplar. Dizin girişi yalnızca başlangıç bloğu numarasını ve blok sayısını saklar.

Özellik	Açıklama
Depolama	Başlangıç bloğu + uzunluk yeterli
Sıralı Erişim	Mükemmel (bloklar bitişik)
Rastgele Erişim	İyi (offset hesaplanabilir)
Dış Parçalanma	Zamanla ciddi boyutta oluşur
Ekleme (Append)	Sona boş alan yoksa dosya taşınmalı

⚠ Görsel: Bitişik Tahsis

Disk: [A A A | B B | C C C C | boş | D D | boş boş | E E E]

D ve F dosyaları silindiğinde: [A A A | B B | C C C C | BOŞLUK | BOŞLUK | E E E]

→ Dış parçalanma (External Fragmentation) oluştu: E'nin yanına büyük bir dosya sığmayabilir!

5.2 Bağlı Liste Tahsisi (Linked Allocation)

Her dosya, disk bloklarının bağlı listesidir. Her blokun sonunda bir sonraki bloğun işaretçisi bulunur. Dizin girişi yalnızca ilk bloğu işaret eder.

Özellik	Açıklama
Depolama	Her blok: veri + next pointer
Sıralı Erişim	Yavaş (her blok okunmalı)
Rastgele Erişim	Çok yavaş (baştan tarama)
Dış Parçalanma	YOK — bloklar dağınık olabilir
Ekleme (Append)	Kolay — herhangi bir boş blok bulunur
Blok boyutu	Pointer için alan harcanır

[LINKED ALLOCATION]

Disk: [217] → [618] → [339] → [NULL]

Dizin girdisi: test | başlangıç: 217

Blok 217: [veri...] [next: 618]

Blok 618: [veri...] [next: 339]

Blok 339: [veri...] [next: NULL] ← son blok

5.3 FAT (File Allocation Table)

FAT, bağlı liste tahsisinin geliştirilmiş versiyonudur. Bağlantı bilgisi (pointer) her bloktan alınıp merkezi bir tabloya (FAT) taşınır. FAT, tüm disk için bellekte tutulur.

[FAT]

Disk boyutu: 200 GB, Blok boyutu: 1 KB
 Blok sayısı: $200 * 1024 * 1024 = 209,715,200$ blok
 Her FAT girişi 4 byte (32-bit adres için)
 FAT boyutu: $209,715,200 \times 4 = \sim 800$ MB (!)
 FAT tablosu örneği:
 Blok: 0 1 2 3 4 5 6 7 8 ...
 Değer: - - END 7 - 6 END 8 END ...
 A = {2, 7, 8} → 2→7→END
 B = {5, 6} → 5→6→END

Özellik	Değerlendirme
Disk I/O	FAT bellekte olduğundan disk erişimi gerektirmez
Rastgele Erişim	Hâlâ yavaş — FAT üzerinden tarama gerekir
Bellek Kullanımı	200 GB disk için ~800 MB RAM gerekir — çok fazla!
Kullanım	MS-DOS, FAT12/16/32, USB sürücüler

5.4 İndeksli Tahsis (Indexed Allocation) ve I-node

Tüm blok işaretçileri, **index block** (indeks bloğu) adı verilen özel bir blokta toplanır. Unix sistemlerinde bu yapıya **i-node (index node)** denir. Dosya açıkken i-node bellekte tutulur; kapalıyken diske yazılır.

[I-NODE STRUCTURE]

I-Node içeriği:
 mode : dosya türü + izinler
 uid/gid : sahip bilgisi
 size : dosya boyutu (byte)
 atime : son erişim zamanı
 mtime : son değiştirilme zamanı
 nlink : hard link sayısı
 direct[0..11] : 12 adet doğrudan blok işaretçisi
 indirect : tek dolaylı blok işaretçisi (single indirect)
 double_indirect: çift dolaylı blok işaretçisi
 triple_indirect: üçlü dolaylı blok işaretçisi

Çok Katmanlı İndirection (Multi-level Indirection)

Büyük dosyaları desteklemek için i-node çok seviyeli indirection (dolaylama) kullanır. Blok boyutu 4 KB, işaretçi boyutu 4 byte olduğunda her indeks bloğu 1024 işaretçi tutar:

Seviye	Formül	Maksimum Alan
Direct (12 blok)	$12 \times 4 \text{ KB}$	48 KB
Single Indirect	$(4K/4) \times 4 \text{ KB} = 1024 \times 4 \text{ KB}$	4 MB
Double Indirect	$1024 \times 1024 \times 4 \text{ KB}$	4 GB
Triple Indirect	$1024 \times 1024 \times 1024 \times 4 \text{ KB}$	4 TB
Toplam (yaklaşık)	Direct + Single + Double + Triple	~4 TB

'Her sorun başka bir indirection katmanıyla çözülür' — David Wheeler

Bu ilke, i-node tasarımının özüdür. PDP-11 Unix'te üçlü dolaylama vardı. Bloklar 4 KB'a çıkınca gerek kalmadı. Ama diskler büyüdü ve bugün yine kullanılıyor.

Yöntem	Avantaj	Dezavantaj	Kullanılan Sistem
Bitişik	Hızlı seq/random erişim Basit izin girdisi	Dış parçalanma Ekleme güç	CD-ROM, eski sistemler
Bağlı Liste	Dış parçalanma yok Ekleme kolay	Yavaş random erişim Pointer israfı	Eski DOS
FAT	Bağlı liste + bellek	Bellek kullanımı yüksek Hâlâ yavaş	MS-DOS, FAT32, USB
I-node (Indexed)	Random erişim hızlı Dış parçalanma yok Esnek	Küçük dosyalarda ek yük	Unix/Linux, NTFS

6. Unix V7 ve Modern Unix Dosya Sistemi

6.1 Dosya Sisteminin Bileşenleri

Boot Record	Disk başındaki MBR (Master Boot Record); BIOS tarafından önyükleme için kullanılır. Disk bölüm tablosu (partition table) burada saklanır.
Superblock	Dosya sisteminin temel parametrelerini içerir: i-list'in konumu, blok boyutu, toplam blok sayısı, 'clean shutdown' bayrağı.
I-List	I-node'ların disk üzerindeki dizisidir. I-node numarası bu dizideki indekstir. Modern sistemlerde silindir gruplarına dağıtılmıştır.
Serbest Liste (Freelist)	Boş blokların ve boş i-node'ların takip edildiği yapıdır.
Veri Alanı (Data Area)	Gerçek dosya içeriğinin saklandığı bloklar.

Disk Düzeni (Disk Layout)

[Boot Block | Super Block | I-List | Data Blocks ...]

Modern ext2/3/4'te: Her cylinder group kendi i-list parçasına, serbest listesine ve veri bloklarına sahiptir.

Bu yapı yerellik (locality of reference) sağlar — dosya ve i-node'u birbirine yakın tutulur → disk kolu hareketi azalır.

6.2 I-Node Detayları

I-node, Unix'te bir dosyanın 'kimliği'dir. Dizin girdisi yalnızca bir isim ve i-node numarası içerir. Tüm önemli bilgi i-node'dadır.

- **I-list:** I-node'ların sabit boyutlu dizisidir. I-node numarası = bu dizideki indeks.
- **Dosyalar isimden değil, i-node'dan tanımlanır.** Bir dosyanın birden fazla ismi (hard link) olabilir.
- **I-node'da saklanan bilgi:** stat yapısındaki tüm alanlar + blok adresleri.
- **I-node numarasının kendisi** i-node'da saklanmaz — çünkü zaten dizi indeksidir.

6.3 Dosya İşlemleri

Dosya Açma (open)

Bir dosya açıldığında işletim sistemi aşağıdaki adımları gerçekleştirir:

[OPEN SYSCALL]

1. Yol adı (path name) bileşen bileşen çözümlenir:
/usr/ast/mbox → kök / → usr → ast → mbox
2. Her bileşen için: mevcut izin okunur, isim aranır → i-node no alınır
3. Son bileşenin i-node'u belleğe okunur (zaten yüklüyse referans sayacı ++)
4. Dosya tablosu girdisi (file table entry) oluşturulur
5. Sürecin open file tablosuna eklenir
6. Dönüş değeri: file descriptor (fd) – dosya tablosuna indeks

Dosya Okuma (read)

[READ SYSCALL]

1. Mevcut byte offset → blok numarasına çevir:
blok_no = offset / blok_boyutu
blok_içi_offset = offset % blok_boyutu
2. İlgili bloğu diskten oku (buffer cache'e bak önce!)
3. Gerekli byte'ları kullanıcı tamponuna kopyala
4. Mevcut offset'i güncelle
5. Opsiyonel: Sıralı erişim tespit edilirse bir sonraki bloğu önceden oku (read-ahead / prefetch)

Dosya Yazma (write)

[WRITE SYSCALL]

1. Mevcut bloğun ortasına yazıyorsak: bloğu önce oku
2. Değilsek (yeni blok): freelist'ten blok al + i-node'a ekle
3. Veriyi kullanıcı tamponundan buffer'a kopyala
4. Buffer'ı 'dirty' (kirli) olarak işaretle
5. File offset'i güncelle
→ Yazma asenkron: veri hemen diske gitmez, buffer cache'de bekler

Seek (lseek)

lseek() yalnızca mevcut byte offset'i değiştirir. Disk kolunu fiziksel olarak hareket **ettirmez**. Çok görevli sistemlerde bu zaten anlamsız olurdu — başka süreçler de diski kullanıyor.

Dosya Kapatma (close)

[CLOSE SYSCALL]

1. I-node'un referans sayacını azalt (decrement)
 2. Dosya tablosu girdisini sil
 3. I-node'un link sayacı = 0 ise → dosya silinmiş demektir
→ Tüm blokları freelist'e iade et
- NOT: Açık bir dosyayı unlink edebilirsiniz;
gerçekten silinmesi için hem link count hem ref count sıfır olmalıdır.

Hard Link Oluşturma (link)

[LINK SYSCALL]

- ```
link('dosya.txt', 'alias.txt');
```
1. Yeni dizin girdisi oluştur: (alias.txt, inode\_no)
  2. Inode\_no, mevcut dosyanın i-node numarasıdır
  3. I-node'un nlink (hard link sayısı) değerini artır
- Artık iki isim, aynı i-node'u (dolayısıyla aynı veriyi) gösterir.  
İkisi de 'eşit' — biri 'orijinal' biri 'kopya' değil!

## Hard Link Silme (unlink)

### [ UNLINK SYSCALL ]

- ```
unlink('alias.txt');
```
1. Dizin girdisini sil
 2. I-node'un nlink sayacını azalt
 3. Eğer nlink == 0 VE referans sayısı == 0:
→ Tüm disk bloklarını freelist'e iade et
→ I-node'u boş olarak işaretle (link count = 0)
- Eğer nlink == 0 ama dosya hâlâ açıksa (ref > 0):
→ Bloklar close() çağrıldığında serbest bırakılır

6.4 Yol Adı Çözümleme

Unix'te her sürecin iki önemli dizin özelliği vardır: **geçerli kök (current root)** ve **geçerli çalışma dizini (current working directory)**.

- **Mutlak yol (Absolute path):** '/' ile başlar → geçerli kökten itibaren çözülür.
- **Görelî yol (Relative path):** '/' ile başlamaz → geçerli çalışma dizininden çözülür.

[PATH RESOLUTION]

Örnek: /usr/ast/mbox yolu çözümleme

1. '/' kök dizini oku → inode 1
 2. 'usr' ara → inode 6
 3. inode 6 bir dizin mi? Evet → devam
 4. 'ast' ara → inode 26
 5. 'mbox' ara → inode 60
 6. Sonuç: inode 60 → dosya!
- '.' → mevcut dizinin inode numarasıdır
 '..' → üst dizinin inode numarasıdır

⚠ Sembolik Link ve Döngü Sorunu

Sembolik link (soft link), başka bir yola işaret eden özel bir dosyadır. Yol çözümlemesi sembolik linki takip eder. Bu, **döngülere (loops)** yol açabilir: A → B → C → A gibi. Kernel bu durumu önlemek için takip sayısını sınırlar (Linux'ta genellikle 40 hop limiti vardır).

6.5 Dizin Oluşturma ve Silme

[MKDIR / RMDIR]

- mkdir('newdir'); // Dizin oluşturma
1. Normal dosya oluşturma gibi
 2. Kernel otomatik olarak '.' ve '..' girdilerini yazar
 3. Üst dizinin link sayısını artır ('..' yeni dizinden üste işaret eder)
- rmdir('newdir'); // Dizin silme
1. Dizin yalnızca '.' ve '..' içerdiğini doğrula
 2. Normal dosya silme gibi sil
 3. Üst dizinin link sayısını azalt
 4. Dikkat: Geçerli çalışma dizinini bile silebilirsiniz!

7. Dizinler (Directories)

Dizinler, dosya adlarını i-node numaralarına eşleyen özel bir dosya türüdür. Hiyerarşik dizin yapısı, modern dosya sistemlerinin temel organizasyon ilkesidir.

7.1 Tek Seviyeli Dizin (Single-Level Directory)

Eski sistemlerde tüm dosyalar tek bir dizinde saklanıyordu. Bu yaklaşım kullanıcı sayısı arttıkça hızla yetersiz kaldı.

⚠ Örnek: Tek seviyeli dizin

[dosya1.txt | dosya2.c | mektup.doc | resim.bmp | ...]

Sorun: 100 kullanıcı, her biri 50 dosya → 5000 dosya tek yerde! İsim çakışmaları kaçınılmaz.

7.2 Hiyerarşik Dizin Sistemi

Modern sistemlerde dizinler ağaç yapısında organize edilir. Her kullanıcı kendi alt ağacına sahiptir.

[UNIX DIRECTORY TREE]

```

/
├── usr/
│   ├── ast/
│   │   ├── mbox
│   │   └── grants/
│   └── jim/
│       └── foo/
├── bin/
│   ├── ls
│   └── cp
└── etc/
    └── passwd
  
```

7.3 Dizin Girdisi Yapıları

Dizin girdisi iki temel şekilde tasarlanabilir:

Yaklaşım	İçerik	Avantaj	Dezavantaj
Sabit boyutlu girdiler	İsim + disk adresi + özellikler	Basit arama	İsim uzunluğu kısıtlı
I-node referansı	İsim + i-node numarası	Esneklik, hard link kolay	Ek I/O (i-node okunmalı)

7.4 Uzun Dosya Adları

Uzun değişken uzunluklu dosya adlarını saklama için iki yöntem kullanılır:

- **Satır içi (In-line):** Uzunluk alanı + isim baytları dizin girdisinin içinde. Boş alan doldurulur (padding).
- **Heap'te:** Dizin girdisi sabit boyutlu kalır; isim, ayrı bir heap alanında saklanır. Dizin girdisi heap'teki offset'i işaret eder.

7.5 Dizin İşlemleri (Directory Operations)

opendir()	Dizini okumak için açar
closedir()	Dizini kapatır
readdir()	Bir sonraki dizin girdisini okur
mkdir()	Yeni dizin oluşturur
rmdir()	Boş dizini siler
rename()	Dizin veya dosyayı yeniden adlandırır
link()	Hard link oluşturur
unlink()	Dizin girdisini kaldırır

8. Paylaşılan Dosyalar ve Hard Link

Birden fazla kullanıcının aynı dosyaya farklı isimlerle erişmesi gerektiğinde iki temel mekanizma kullanılır: **hard link** ve **symbolic link**.

Özellik	Hard Link	Symbolic Link (Soft Link)
Nasıl çalışır	Aynı i-node'u paylaşır	Hedef yol adını içeren özel dosya
I-node	Aynı i-node numarası	Farklı i-node numarası
Orijinal silinince	Veri hâlâ erişilebilir (nlink>0)	Link kırılır (dangling link)
Dosya sistemi sınırı	Aynı FS içinde olmalı	Farklı FS'e, hatta ağa işaret edebilir
Dizin linki	Kernel genellikle izin vermez	İzin verilir (dikkatli kullanılmalı)
Boyut	0 ek alan	Yol adı kadar alan

Paylaşılan Dosya Senaryo Örneği

Senaryo: B kullanıcısı, A kullanıcısının /usr/A/shared.txt dosyasına erişmek istiyor.

(a) Önceki durum: /usr/A/shared.txt → inode 72

(b) Hard link: /usr/B/shared.txt → inode 72 (nlink=2)

(c) A dosyayı silerse: /usr/A/shared.txt yok, ama /usr/B/shared.txt → inode 72 hâlâ erişilebilir

Sorun: A'nın eski bloklarını hâlâ sayan disk kota sistemi için sorun çıkabilir.

9. Journaling ve Çökme Kurtarma

Dosya sistemi işlemleri birden fazla disk yazması gerektirir. Sistem bu yazmaların ortasında çökerse dosya sistemi tutarsız hale gelebilir.

9.1 Sorun: Atom Olmayan İşlemler

Örnek: Unix'te bir dosyayı silmek için üç ayrı işlem gerekir:

[CRASH SCENARIO]

1. Dizin girdisini kaldır → directory write
2. I-node'u serbest bırak → inode bitmap write
3. Disk bloklarını freelist'e iade → block bitmap write

Eğer sistem 1. adımdan sonra çökse:

- Dizin girdisi yok, ama inode ve bloklar kullanılıyor görünüyor
- Hem 'kayıp' (lost) hem 'sızdırılmış' (leaked) kaynaklar!

9.2 Çözüm 1: Yazma Sıralaması (Write Ordering)

Disk'in her zaman güvenli bir durumda olmasını garanti etmek için yazma sıralaması yapılır. Temel ilke: Dizin girdisini yazmadan önce metadata'yı yaz.

9.3 Çözüm 2: fsck ile Onarım

Önyükleme sırasında (temiz kapatılmamışsa) **fsck** (Unix) veya **scandisk** (Windows) çalıştırılır.

[FSCK]

fsck algoritması:

Her blok için iki sayaç tablo:

Tablo 1: Blok herhangi bir dosyada kaç kez geçiyor?

Tablo 2: Blok freelist'te kaç kez görünüyor?

Tutarlı durum: her blok (tablo1=1, tablo2=0) VEYA (tablo1=0, tablo2=1)

Sorunlu durumlar:

- (b) Eksik blok: tablo1=0, tablo2=0 → freelist'e ekle
- (c) Freelist'te çift blok: tablo1=0, tablo2=2 → bir kez tut
- (d) Veri bloğu çift kullanım: tablo1=2, tablo2=0 → bir kopyasını ver

9.4 Çözüm 3: Journaling File Systems

Verinin üzerine doğrudan yazmak yerine, değişiklikler önce bir **journal (günlük)** alanına eklenir. Journal'dan ana yapıya geçiş atomik biçimde yapılır.

[JOURNALING]

Journal'lı yazma akışı:

1. Transaction başlangıcı journal'a yaz (BEGIN)
2. Tüm değişiklikleri journal'a yaz
3. Transaction sonu işaretle (COMMIT)
4. Ana dosya sistemine değişiklikleri uygula
5. Journal girdisini temizle (CHECKPOINT)

Çökme durumunda:

- COMMIT görülürse: değişiklikleri tekrar uygula (redo)
- COMMIT görülmezse: değişiklikleri geri al (undo/ignore)

Dosya Sistemi	Journal Türü	Açıklama
ext3/ext4 (Linux)	Metadata veya Full	Yaygın kullanım; ordered mode varsayılan
NTFS (Windows)	Metadata	Log-structured MFT güncellemeleri
XFS	Metadata	Yüksek performanslı, 64-bit FS
ZFS	Copy-on-Write	Journal yerine CoW kullanır; çok güvenli
APFS (Apple)	Copy-on-Write	Atomik işlemler, şifreleme, snapshot

× Modern Diskler ve Tehlike

Modern diskler dahili buffer (2-16 MB) kullanır ve yazmaları seek süresini minimize etmek için yeniden sıralar. Güç kesildiğinde buffer'daki veri **kaybolabilir**. Bu, journaling'in doğruluğunu tehdit eder! Güvenilir journaling için aygıtın write barrier (yazma bariyeri) desteklemesi gerekir.

10. Sanal Dosya Sistemi (VFS)

Modern işletim sistemleri birden fazla dosya sistemi türünü aynı anda desteklemek zorundadır (ext4, NTFS, FAT32, NFS, tmpfs...). **VFS (Virtual File System)** bu farklılıkları gizleyerek uygulamalara tek tip bir arayüz sunar.

[VFS ARCHITECTURE]

Uygulama (Application)

↓ open(), read(), write(), close()

VFS Katmanı (Virtual File System Layer)

↓

ext4 Driver

NTFS Driver

NFS Driver

↓

↓

↓

Yerel Disk

Yerel Disk

Ağ Sunucusu

VFS, her dosya sistemi türü için aynı arayüzü sağlar. Her somut dosya sistemi, bu arayüzü implemente eden bir sürücü sağlar. Temel VFS nesneleri:

superblock	Bağlı dosya sistemi hakkında bilgi; bağlama/çözme işlemleri
inode	Bireysel dosya hakkında bilgi; dosya operasyonları
dentry	Dizin girdisi; yol çözümleme için önbelleklenir
file	Açık dosya; okunan/yazılan süreçle ilişkilendirilir

11. Disk Alan Yönetimi ve Performans

11.1 Blok Boyutu Seçimi

Dosya sistemi blok boyutu, performans ve alan verimliliği arasında kritik bir denge noktasıdır.

Blok Boyutu	Veri Aktarım Hızı	Alan Verimliliği	Yorum
512 B	Düşük	Yüksek (%100 yakın)	Çok küçük dosyalar için iyi
1 KB	Orta	Yüksek	Eski ext2 varsayılını
4 KB	Yüksek	İyi	Linux ext4 varsayılını
64 KB	Çok yüksek	Düşük	Büyük dosyalar için iyi

⚠ İç Parçalanma (Internal Fragmentation)

4 KB blok boyutuyla 1 byte'lık bir dosya, 4 KB disk alanı kaplar. Kalan 4095 byte israf edilir — buna iç parçalanma denir. Küçük dosyalar yaygınsa, blok boyutunu küçük seçmek alan verimliliğini artırır.

Gerçek: Dosyaların çoğunluğu 4 KB'den küçüktür!

11.2 Boş Blok Takibi

Dosya sistemi, hangi blokların boş olduğunu takip etmek için iki yöntem kullanır:

Yöntem	Yapı	Avantaj	Dezavantaj
Bağlı Liste (Linked List)	Boş bloklar birbirine bağlı	Basit uygulama	Blok başına işaretçi bloğu gerekli
Bit Haritası (Bitmap)	Her blok için 1 bit (0=boş, 1=dolu)	Hızlı tarama, küçük alan	Büyük disklerde bellek gerekir

[BITMAP]

Bitmap örneği (16 blok):

Blok: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Durum: 1 1 0 1 0 0 1 1 0 0 1 0 1 1 0 0

1 = dolu, 0 = boş

Blok 2, 4, 5, 8, 9, 11, 14, 15 boş.

11.3 Disk Kota Yönetimi (Disk Quotas)

Disk kotası, her kullanıcının kullanabileceği maksimum disk alanını ve dosya sayısını sınırlar. Çok kullanıcıli sistemlerde vazgeçilmezdir.

Soft Limit	Aşılabilir (kısa süre için); uyarı verilir
Hard Limit	Kesinlikle aşılamaz
Kota Tablosu	Her kullanıcı için blok sayısı ve dosya sayısı takip edilir
Kontrol	Dosya açıldığında kullanıcının kota tablosuna girdi eklenir

11.4 Performans İyileştirmeleri

Buffer Cache (Tampon Önbellek)

Disk erişimi, bellek erişiminden çok daha yavaştır. Buffer cache, son zamanlarda kullanılan disk bloklarını bellekte tutar.

[BUFFER CACHE]

Buffer Cache Yapısı:

Hash tablosu: (aygıt, blok_no) → buffer adresi

LRU listesi: En az yakın zamanda kullanılanlar önce çıkar

Blok okunurken:

1. Hash tablosunda ara
2. Bulunduysa: cache hit → LRU listesini güncelle, ver
3. Bulunmadıysa: cache miss → diskten oku, cache'e ekle

Değiştirilmiş LRU stratejisi:

- 'Yakında tekrar kullanılacak mı?' sorusunu dikkate al
- I-node blokları gibi kritik bloklar → hemen diske yaz
- Sıradan veri blokları → gecikmeli yaz (lazy write)

Read-Ahead (Önceden Okuma)

Sıralı erişim tespit edildiğinde, henüz talep edilmemiş bir sonraki blok önceden okunur. Bu, I/O gecikmesini örtbas eder (overlap: hesaplama + I/O aynı anda).

Disk Kolu Hareketini Azaltma

Disk kolu hareketi (seek), en büyük gecikme kaynağıdır. İki temel yaklaşım:

- **Silindir Grupları (Cylinder Groups):** Bir dosyanın i-node'u ve veri blokları aynı silindir grubuna yerleştirilir. Kısa seek'ler → hızlı erişim.

- **I-node Dağılımı:** Eski sistemlerde i-list diskin başındaydı; veri blokları sonundaydı. Bu büyük seek'lere yol açıyordu. Silindir grupları bu sorunu çözdü.

12. Yedekleme (Backup) Stratejileri

Yedekleme, iki tür felaketten korunmak için yapılır:

- **Donanım/Yazılım felaketi:** Disk arızası, yangın, sel...
- **Kullanıcı hatası:** Yanlışlıkla silinen/değiştirilen dosyalar.

12.1 Fiziksel Dump vs Mantıksal Dump

Özellik	Fiziksel Dump (Physical)	Mantıksal Dump (Logical)
Yöntem	Blok 0'dan itibaren tüm bloğu kopyala	Dizin ağacını gezerek sadece dosyaları kopyala
Hız	Hızlı	Daha yavaş
Boş bloklar	Yedeklenir (israf)	Yedeklenmez
Artımlı yedek	Desteklemez	Destekler (sadece değişenler)
Kötü bloklar	Kopyalanabilir — tehlikeli!	Atlanır
Geri yükleme	Aynı geometrili diske olmalı	Herhangi bir sisteme geri yüklenir

12.2 Artımlı Dump Algoritması

Unix'te kullanılan mantıksal dump algoritması 4 aşamada çalışır:

[DUMP ALGORITHM]

```
Aşama 1: Son dump'dan beri değiştirilen tüm dosyaları işaretle
          Her dizini de işaretle (değişmiş olsun ya da olmasın)
Aşama 2: Değişmiş dosya içermeyen dizinlerin işaretini kaldır
          (Değişmemiş alt ağaçları atla)
Aşama 3: İşaretili dizinleri dump et (i-node numarası sırasıyla)
Aşama 4: İşaretili dosyaları dump et
Neden tüm dizinler? Silme işlemleri de kurtarılabilmeli:
          Silinen dosya → üst dizinde değişiklik → dizin dump'a giriyor
```

12.3 Geri Yükleme (Recovery)

[RECOVERY]

1. Hedef diskte boş dosya sistemi oluştur
2. En son tam (level-0) dump'ı geri yükle:
 - a. Önce dizinleri geri yükle
 - b. Sonra dosyaları geri yükle
3. Her artımlı dump için aynı adımları tekrarla
(Eski → Yeni sırasıyla)

Sorunlar:

- Serbest bloklar: Geri yükleme tamamlandıktan sonra yeniden hesapla
- İki dizinde paylaşılan dosya: Bir kez geri yükle

Dump Seviyesi	Ne Yedekler	Açıklama
Level 0	Her şeyi	Tam yedek — tüm dosya sistemi
Level 1	Son Level 0'dan beri değişenler	İlk artımlı yedek
Level 2	Son Level 1'den beri değişenler	İkinci artımlı yedek
Level N	Son Level N-1'den beri değişenler	Kademeli artımlı yedek

13. Dosya Sistemi Tutarlılığı (File System Consistency)

Sistem çöktükten sonra dosya sistemi tutarsız hale gelebilir. **fsck** (Unix) ve **scandisk/chkdsk** (Windows) bu tutarsızlıkları tespit edip onarır.

13.1 Blok Tutarlılık Kontrolü

[FSCK BLOCK CHECK]

İki sayaç tablosu her blok için:

Tablo 1: Bu blok kaç farklı dosyada geçiyor?

Tablo 2: Bu blok freelist'te kaç kez görünüyor?

Her i-node taranarak blok listeleri güncellenir.

Her freelist girişi taranarak tablo 2 güncellenir.

Tutarlı durum:

Her blok için: (T1=1, T2=0) VEYA (T1=0, T2=1)

Sorunlar ve çözümler:

- (a) Tutarlı: Normal durum
- (b) Eksik blok (T1=0, T2=0): Freelist'e ekle
- (c) Freelist'te çift (T1=0, T2=2): Bir kopyasını kaldır
- (d) Veri bloğu çift kullanım (T1=2, T2=0): Bir kopyaya ver, uyar

13.2 Dizin Tutarlılık Kontrolü

fsck ayrıca dizin yapısını da kontrol eder:

- Her i-node için link sayacı ile gerçek dizin girdisi sayısı karşılaştırılır.

- Eşleşmiyorsa, i-node'daki link sayacı güncellenir.
- Link sayacı 0 ama i-node kullanılıyorsa → dosya lost+found'a taşınır.

14. Özel Dosya Sistemleri

14.1 ISO 9660 (CD-ROM)

CD-ROM'lar için standart dosya sistemidir. Temel özellikler:

Yapı	16 blok başlangıç; Primary Volume Descriptor
Dosya adı	8.3 format, büyük harf, sınırlı karakter seti
Hiyerarşi	En fazla 8 seviye dizin derinliği
Rock Ridge	POSIX uzantıları: uzun isimler, symlink, izinler
Joliet	Unicode, uzun isimler, 8'den fazla seviye

14.2 MS-DOS / FAT Dosya Sistemi

Özellik	FAT12	FAT16	FAT32
Blok başına bit	12 bit	16 bit	28 bit (gerçekte)
Max bölüm	~16 MB	2 GB	2 TB
Dosya adı	8.3	8.3	Uzun (LFN)
Kullanım yeri	Disket	Eski HDD	USB, SD kart

Windows'ta Uzun Dosya Adı Hilesi (LFN)

DOS uyumluluğunu korumak için Windows 98, sahte geçersiz dizin girdileri oluşturur. Bu girdiler uzun adı saklar; DOS bu girdileri geçersiz olarak görüp atlar. Gerçek kısa ada checksum eklenir — kısa ad dosya silinip yeniden oluşturulursa long entry geçersiz kalmasın diye.

14.3 Unix V7 Dosya Sistemi

Bileşen	Boyut / Özellik
Dizin girdisi	14 byte isim + 2 byte i-node no = 16 byte
I-node boyutu	64 byte
Doğrudan bloklar	10 adet
Tek dolaylı blok	1 adet (512 blok/blok)
Çift dolaylı blok	1 adet (512×512 blok)
Max dosya boyutu	~1 GB (1 KB blok ile)

15. Quiz Soruları ve Cevapları

Quiz Hakkında

Aşağıdaki sorular sınav formatında hazırlanmıştır. Her soru için önce kendiniz cevap vermeyi deneyin, ardından cevabı kontrol edin. Sorular dersin tüm konularını kapsamaktadır.

Soru S1

Dosya sistemi tasarımında 'bitişik tahsis' (contiguous allocation) yönteminin temel avantajları ve dezavantajları nelerdir? Hangi kullanım senaryolarında tercih edilir?

✓ Cevap

Avantajlar: (1) Dizin girdisi yalnızca başlangıç bloğu ve uzunluk saklar — basit. (2) Sıralı erişim mükemmeldir: bloklar ardışık olduğundan disk kolu hareketi minimumdur. (3) Rastgele erişim de iyidir: $\text{blok_no} = \text{başlangıç} + \text{offset} / \text{blok_boyutu}$ ile hesaplanır. Dezavantajlar: (1) Dış parçalanma (external fragmentation) zamanla oluşur. (2) Dosya büyürken (append) yeterli bitişik alan bulunamayabilir, dosya taşınmalıdır. Kullanım: CD-ROM gibi salt okunur medyalar için idealdir; dosya boyutları önceden bilinir.

Soru S2

Bir Unix i-node'unda 'double indirect block' (çift dolaylı blok) nasıl çalışır? 4 KB blok boyutu ve 4-byte işaretçi boyutuyla ulaşılabilecek maksimum dosya boyutunu hesaplayın.

✓ Cevap

Single indirect block: i-node → bir blok (1024 işaretçi) → 1024 veri bloğu. Double indirect block: i-node → blok A (1024 işaretçi) → 1024 adet single indirect blok → 1024×1024 veri bloğu. Hesaplama: Direct: $12 \times 4K = 48$ KB. Single: $1024 \times 4K = 4$ MB. Double: $1024 \times 1024 \times 4K = 4$ GB. Triple: $1024 \times 1024 \times 1024 \times 4K = 4$ TB. Toplam ≈ 4 TB (pratikte dosya sistemi sınırlamaları daha küçük limitlere yol açar).

Soru S3

Journaling (günlükleme) dosya sistemleri neden gereklidir? Journaling olmadan bir dosyayı silerken sistem çökmesi durumunda ne olur?

✓ Cevap

Dosya silme 3 ayrı disk yazması gerektirir: (1) Dizin girdisini kaldır, (2) i-node'u serbest bırak, (3) disk bloklarını freelist'e iade et. Bu işlemler atomic (bölünmez) değildir. Eğer sistem (1) sonrasında çökerse: i-node ve bloklar hâlâ kullanılıyor görünür ama dizin girdisi yok → sızdırılmış kaynak (resource leak) + dosya sistemi tutarsızlığı. Journaling: değişiklikler önce journal'a atomik biçimde yazılır. Çöküşten sonra journal replayed edilir → dosya sistemi tutarlı kalır.

Soru S4

FAT (File Allocation Table) sisteminde 200 GB disk ve 1 KB blok boyutu kullanıldığında FAT tablosu kaç byte yer kaplar? Bu sorun nasıl çözülür?

✓ Cevap

Blok sayısı: $200 \text{ GB} / 1 \text{ KB} = 200 \times 1024 \times 1024 = 209,715,200$ blok. Her FAT girişi 4 byte (32-bit adres): $209,715,200 \times 4 = \sim 838 \text{ MB}$. Bu, FAT tablosunun bellekte tutulması için 838 MB RAM gerektirir — kabul edilemez! Çözümler: (1) Blok boyutunu büyütme (örn. 4 KB → 4x daha küçük FAT). (2) FAT32'de 28-bit adres → maksimum 2 TB bölüm ve daha küçük tablo. (3) Modern sistemler i-node tabanlı yaklaşım kullanır.

Soru S5

Unix'te bir dosyayı unlink() yaptıktan hemen sonra program çöküşe uğrarsa ve dosya hâlâ açıksa ne olur? fsck bu durumu nasıl ele alır?

✓ Cevap

Unlink() yapılırken link count sıfıra düşer. Ancak dosya hâlâ açık olduğundan in-memory reference count > 0'dır. Dosya gerçekte silinmez — bloklar close() çağrıldığında (veya süreç sonlandığında) serbest bırakılır. Sistem çöküşünde: i-node bellekte kalmaz, bloklar serbest bırakılmamış olabilir. fsck: link count = 0 ama bloklar referanslı gördüğünde → dosyayı lost+found dizinine taşır. Kullanıcı dosyayı oradan kurtarabilir.

Soru S6

Hard link ile symbolic link (soft link) arasındaki temel farkları açıklayın. Dizinlere neden genellikle hard link oluşturulamaz?

✓ Cevap

Hard link: Aynı i-node'u paylaşır. Orijinal silinse bile link count > 0 ise veri erişilebilir. Aynı dosya sistemi içinde olmak zorunda. Symbolic link: Hedef yol adını içeren özel dosya. Orijinal silinince dangling link oluşur. Farklı dosya sistemlerini ve ağı işaret edebilir. Dizin hard link yasağı: Dizin hard linkleri grafta döngüler (cycles) oluşturabilir. Bu, tree walk algoritmalarının (fsck, find, du) sonsuz döngüye girmesine yol açar. Kernel bu riski önlemek için izin hard linklerine izin vermez (yalnızca . ve .. istisnası).

Soru S7

Buffer cache (tampon önbellek) nedir ve LRU stratejisinin değiştirilmiş versiyonu neden gereklidir? Hangi bloklar için farklı davranış sergilenmelidir?

✓ Cevap

Buffer cache: Son zamanlarda kullanılan disk bloklarını bellekte tutan yapı. Disk erişimi yerine RAM'den servis eder → çok daha hızlı. Standart LRU: En az yakın zamanda kullanılan blok önce çıkarılır. Değiştirilmiş LRU gereksinimi: Bazı bloklar yakında tekrar kullanılsa da kritiktir. İ-node blokları, superblock, izin blokları: Dosya sistemi tutarlılığı için kritik. Güç kesilirse bellekte tutmak tehlikeli → hemen diske yazılmalı. Normal veri blokları: Gecikmeli yazma (lazy

write) kabul edilebilir. Bu dengeyi kurmak için blok türüne göre çıkarma politikası belirlenir.

Soru S8

Logical dump (mantıksal yedekleme) algoritmasının 4 aşamasını açıklayın. Neden yalnızca değiştirilen dosyalar değil, tüm izinler de dahil edilir?

✓ Cevap

Aşama 1: Bitmap'te değiştirilen tüm dosyaları ve tüm izinleri işaretler. Aşama 2: Değiştirilmiş dosya içermeyen izinlerin işaretini kaldır. Aşama 3: İşaretli izinleri i-node sıralamasıyla dump et. Aşama 4: İşaretli dosyaları dump et. İzinlerin dahil edilme sebebi: Geri yüklemede doğru izin ağacının kurulabilmesi için tüm ebeveyn izinlerin dump edilmesi şarttır. Ayrıca bir dosya SİLİNMIŞSE üst dizinde değişiklik oluşur → izin dump'a girer → geri yüklemede silme de yansır.

16. Kitap Bölüm Sonu Soruları ve Çözümleri

Modern Operating Systems — Tanenbaum, Chapter 4 (File Systems)

Aşağıdaki sorular, ders kitabının File Systems bölümündeki soru setinden seçilmiş ve kapsamlı biçimde çözülmüştür.

Kitap Sorusu TB-1

Kullanıcıların sabit disklerini satın almamasına rağmen başkasından disk kiralayıp dosya sistemlerini bulutta depolaması, güvenilirlik açısından hangi zorlukları doğurur? Bulut depolamanın avantajları nelerdir?

✓ Çözüm

Zorluklar: (1) Bulut sağlayıcı çökebilir veya iflas edebilir → veri kayıpları. (2) Ağ bağlantısı kesildiğinde verilere erişilemez (yerel diskten farklı). (3) Güvenlik: Veriler üçüncü taraf sunucularında → gizlilik riski. (4) Gecikme (latency): Disk erişimi yerel değil, ağ üzerinden → yavaş. (5) Vendor lock-in: Sağlayıcı değişimi güç olabilir. Avantajlar: (1) Her yerden erişim. (2) Otomatik yedekleme. (3) Ölçeklenebilirlik (storage on demand). (4) Donanım bakımı sağlayıcıda. (5) Coğrafi çoğaltma → afet kurtarma.

Kitap Sorusu TB-2

Unix'te bir dosyanın kopyalanması ve o kopyaya hard link oluşturulması arasındaki farkı açıklayın. Her iki durumda disk alanı kullanımı nasıl değişir?

✓ Çözüm

Kopyalama (cp dosya1 dosya2): Yeni bir i-node oluşturulur. Verinin tüm blokları fiziksel olarak kopyalanır. Artık iki bağımsız dosya var: dosya1 değiştirilse dosya2 etkilenmez. Disk alanı: 2x özgün boyut. Hard link (ln dosya1 dosya2): Yeni bir dizin girdisi oluşturulur, aynı i-node numarası atanır. i-node'un nlink sayacı 1 artar. Fiziksel bloklar kopyalanmaz. dosya1 içeriği değiştirilirse dosya2 de değişmiş görünür (aynı bloklar). Disk alanı: Neredeyse sıfır ek alan (sadece dizin girdisi kadar).

Kitap Sorusu TB-3

Çok kullanıcılu bir sistemde disk kotası (disk quota) neden gereklidir? Soft limit ile hard limit arasındaki fark nedir?

✓ Çözüm

Gereklilik: Disk kotası olmadan bir kullanıcı tüm disk alanını doldurabilir, diğer kullanıcılar dosya yazamaz hale gelir. Sistem güvenliği ve hakkaniyeti için her kullanıcıya sınır belirlenmesi zorunludur. Soft limit: Kullanıcı bu sınırı geçebilir, ancak sistem uyarı verir. Belirli bir süre (grace period, tipik 7 gün) içinde aşılabılır. Süre dolunca soft limit = hard limit gibi davranır. Hard limit: Kesinlikle aşılamaz. Disk yazma işlemi hata ile döner. Bu iki seviyeli yapı, kullanıcılara kısa

vadeli esneklik tanırken sistemin uzun vadeli stabilitesini korur.

Kitap Sorusu TB-4

Contiguous, linked ve indexed olmak üzere üç dosya tahsis yönteminin sıralı (sequential) ve rastgele (random) erişim performansını karşılaştırın.

✓ Çözüm

Contiguous: Sıralı → Mükemmel (bloklar bitişik, tek seek). Rastgele → İyi (başlangıç + offset hesabı). Linked: Sıralı → Yavaş (her blok için disk okuması + pointer takibi). Rastgele → Çok yavaş (n. bloğa ulaşmak için n disk okuması). FAT (Linked+Tablo): Sıralı → Orta (tablo bellekte, disk I/O yok). Rastgele → Orta (tablo taranmalı). Indexed (I-node): Sıralı → İyi (blok adresleri i-node'da). Rastgele → İyi (blok no → dolaysız/dolaylı blok → veri). Büyük dosyalarda double/triple indirect → ekstra disk erişimi. Sonuç: I-node hem esneklik hem de erişim hızı dengesi açısından en iyi çözümdür.

Kitap Sorusu TB-5

Log-structured (journal tabanlı) dosya sistemleri neden modern sistemlerde tercih edilmektedir? Write ordering ile journaling arasındaki farkı açıklayın.

✓ Çözüm

Neden tercih edilir: (1) Güç kesintisi veya çöküşte dosya sistemi tutarlı kalır. (2) fsck çalıştırmadan sistem hızla toparlanır. (3) Büyük veri kümelerinde fsck saatler sürebilir. Write Ordering: Belirli bir sırada yazma zorunluluğu. Örnek: Dizin girdisinden önce i-node'u yaz. Çökme hâlâ sorun oluşturabilir (arada kalabilir). Journaling: Değişikliklerin tümü önce journal'a atomik biçimde yazılır. Sistem journal'ı okuyarak tam olarak nerede kaldığını bilir. COMMIT işareti görülmüşse → uygula (redo). Görülmemişse → atla (undo/ignore). Journaling çok daha güçlü bir garantidir.

Kitap Sorusu TB-6

Bir Unix sisteminde bir dosyayı açtıktan hemen sonra unlink() çağrıldığında ne olur? Bu davranış neden tasarım açısından yararlıdır?

✓ Çözüm

Davranış: unlink() çağrısı link count'u 0'a düşürür. Ancak dosya hâlâ açık olduğundan in-memory reference count > 0'dır. Dosyanın blokları ve i-node'u bu durumda serbest bırakılmaz. close() çağrıldığında (veya süreç sonlandığında) bloklar serbest bırakılır. Bu süre zarfında dosyaya hâlâ okuma/yazma yapılabilir. Dizin girdisi yok olduğundan başka süreçler bu dosyayı bulamaz. Tasarım yararı: Geçici dosya (temp file) idiom'u. Program açar → unlink eder → kullanır → close eder. Dosya asla isim olarak dizinde görünmez → güvenli geçici depolama. Sistem çöküşünde bile otomatik temizlenir (close olmasa da).

Özet: Temel Kavramlar

Kavram	Tanım / Önem
Dosya Sistemi	OS'nin depolama soyutlaması; isim → fiziksel blok eşlemesi
I-node	Unix'te dosya kimliği; tüm metadata + blok adresleri
Hard Link	Aynı i-node, farklı isim; nlink sayacı ile takip
Soft Link	Yol adı içeren özel dosya; dangling link riski
Contiguous	Basit+hızlı ama dış parçalanma; CD-ROM için ideal
FAT	Bağlı liste + bellek tablosu; USB/SD kart için yaygın
I-node (Indexed)	Direct+Indirect bloklar; modern Unix/Linux
Journaling	Atomik günlükleme; fsck gerektirmeden kurtarma
VFS	Çoklu FS türleri için tek tip soyutlama katmanı
Buffer Cache	Disk I/O'yu azaltan bellek önbellegi; LRU politikası
fsck	Çöküş sonrası tutarlılık denetimi ve onarımı
Logical Dump	4 aşamalı artımlı yedekleme algoritması
Bitmap	Boş blok takibi; hızlı ve kompakt veri yapısı

Bu doküman, CSE 312 İşletim Sistemleri dersinin Dosya Sistemleri (File Systems) konusunu kapsamlı biçimde ele almaktadır. Modern Operating Systems (Tanenbaum) ders kitabına dayalı hazırlanmıştır. Spring 2026.